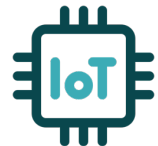
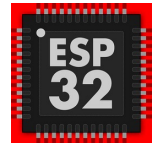


Node-RED + ESP32: DHT11 & Dashboard

Objectives:

To display temperature and humidity readings from DHT11 on the Node-RED with:

- i. Serial node to debug node
- ii. Serial node to function node to 2 debugs (1ESP32 + 2 outputs)
- iii. Serial node to function node to 3 debugs (1ESP32 + 3 outputs)
- iv. Serial node to function node and dashboard with 2 gauges

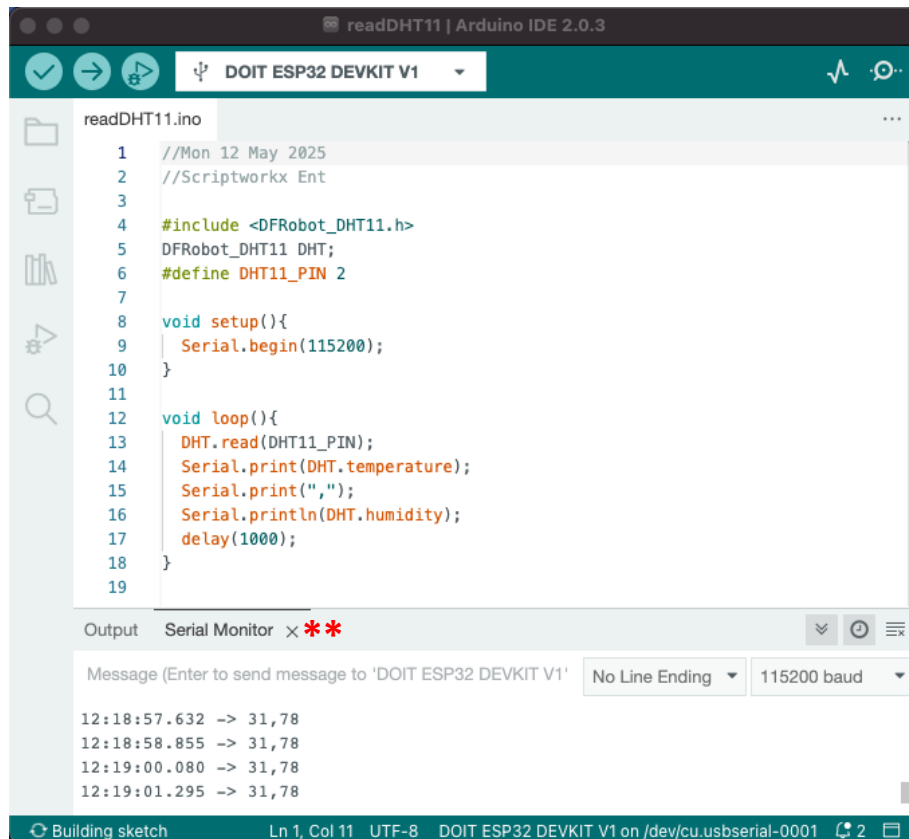


UniKL BMI Comm Tech Section
OCT 2025

i.Upload DHT11 sketch to ESP32

```
1 //Mon 12 May 2025
2 //Scriptworkx Ent
3
4 #include <DFRobot_DHT11.h>
5 DFRobot_DHT11 DHT;
6 #define DHT11_PIN 2
7
8 void setup(){
9   Serial.begin(115200);
10 }
11
12 void loop(){
13   DHT.read(DHT11_PIN);
14   Serial.print(DHT.temperature);
15   Serial.print(",");
16   Serial.println(DHT.humidity);
17   delay(1000);
18 }
```

i. Verify the output by using Serial Monitor



The screenshot shows the Arduino IDE 2.0.3 interface. The top toolbar includes buttons for checking, compiling, uploading, and opening the serial monitor. The dropdown menu shows 'DOIT ESP32 DEVKIT V1'. The main editor displays the code for 'readDHT11.ino'. The code includes comments, library includes, pin definitions, and functions for setup and loop. The Serial Monitor is open at the bottom, showing the output of the code. The output shows four lines of data, each with a timestamp and a comma-separated pair of values (31,78).

```
readDHT11.ino
1 //Mon 12 May 2025
2 //Scriptworkx Ent
3
4 #include <DFRobot_DHT11.h>
5 DFRobot_DHT11 DHT;
6 #define DHT11_PIN 2
7
8 void setup(){
9   Serial.begin(115200);
10 }
11
12 void loop(){
13   DHT.read(DHT11_PIN);
14   Serial.print(DHT.temperature);
15   Serial.print(",");
16   Serial.println(DHT.humidity);
17   delay(1000);
18 }
19
```

Output Serial Monitor ***

Message (Enter to send message to 'DOIT ESP32 DEVKIT V1') No Line Ending 115200 baud

12:18:57.632 -> 31,78
12:18:58.855 -> 31,78
12:19:00.080 -> 31,78
12:19:01.295 -> 31,78

Building sketch Ln 1, Col 11 UTF-8 DOIT ESP32 DEVKIT V1 on /dev/cu.usbserial-0001 2

```
var output = msg.payload.split(",");
msg1={};
msg2={};
var read1 = parseInt(output[0]);
var read2 = parseInt(output[1]);
```

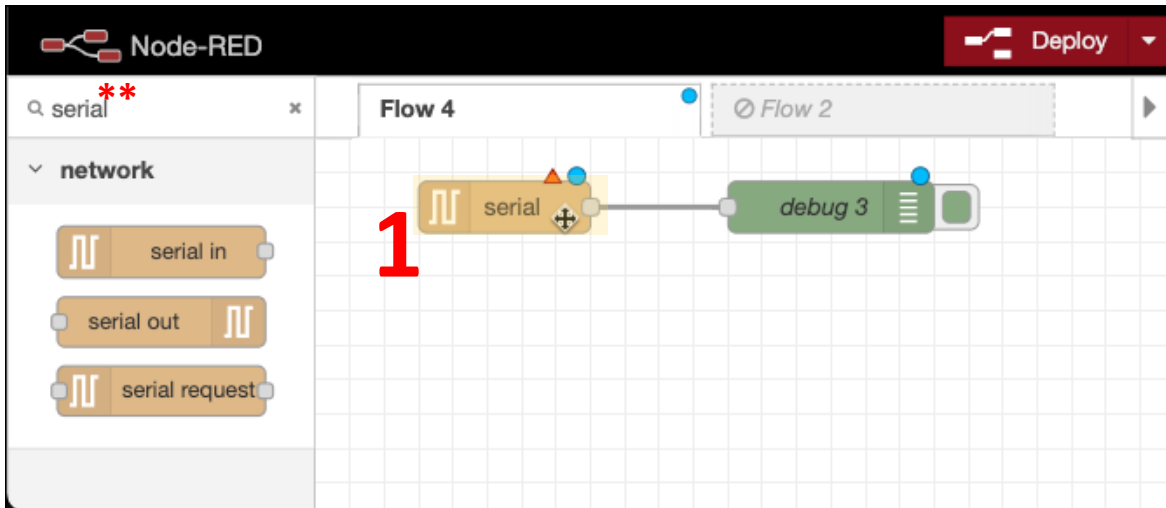
```
msg1 = {payload : read1};
msg1.topic = 'Temperature';
```

```
msg2 = {payload : read2};
msg2.topic = 'Humidity';
```

```
return [msg1,msg2];
```

****Close the Serial Monitor before coding the Node-RED**

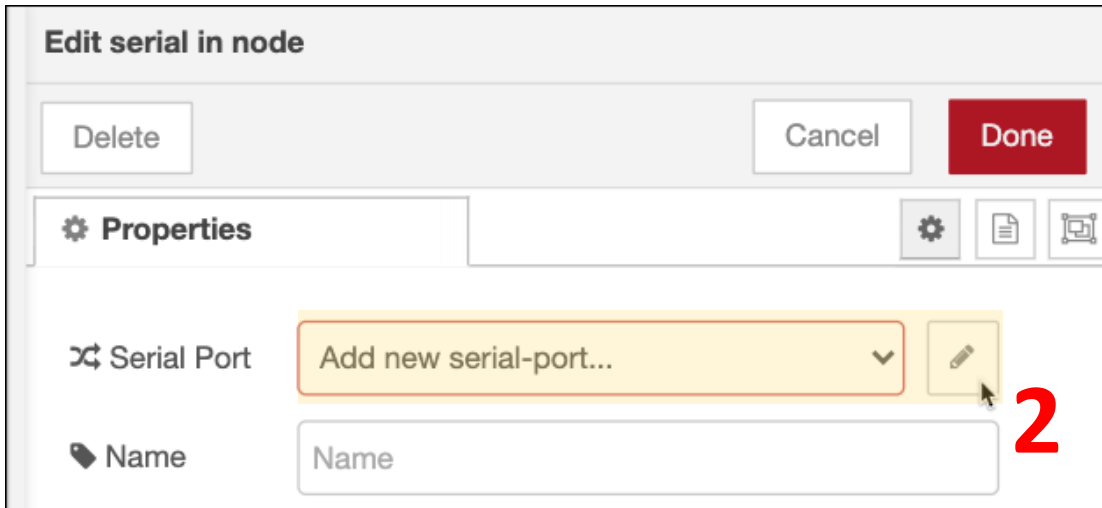
iii. Serial node to debug node



1. Double click **serial** node to configure.
2. Click **pencil icon** to Edit serial-port node.
3. Click **browse icon** and select the port your EPS32 is connected to.
4. Change the **baud rate** to **115200** or refer to ESP32 Serial.begin().
5. Click **Update button**.
6. Click **Done button**.
7. Click **Deploy button**.

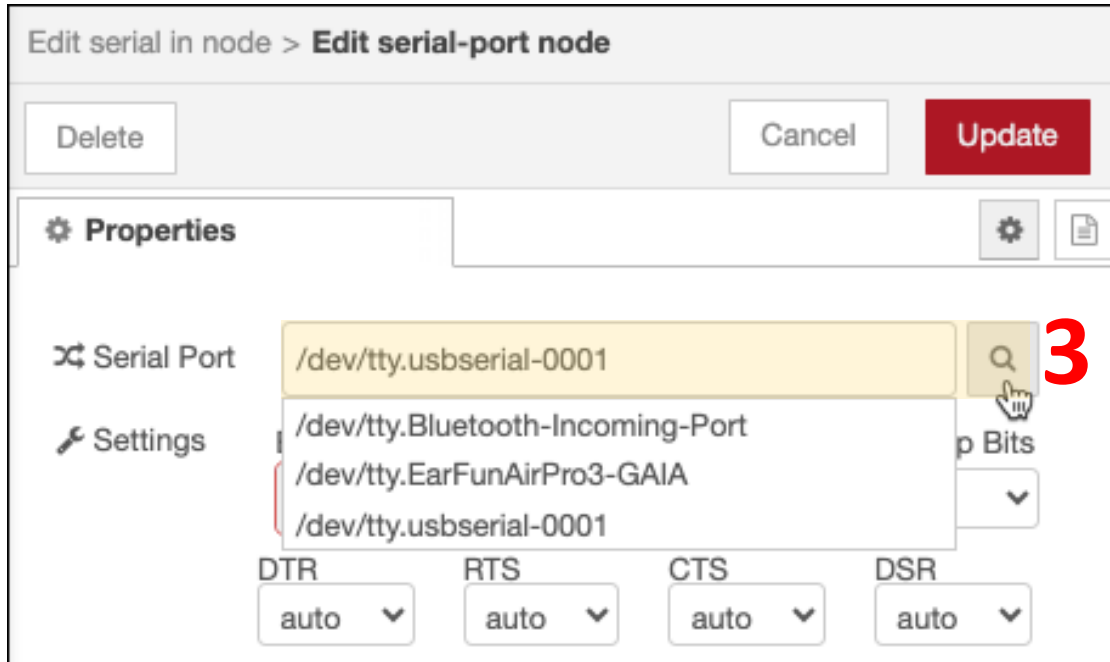
****Palette: node-red-node-serialport
ver 0.14.1**

iii. Serial node to debug node



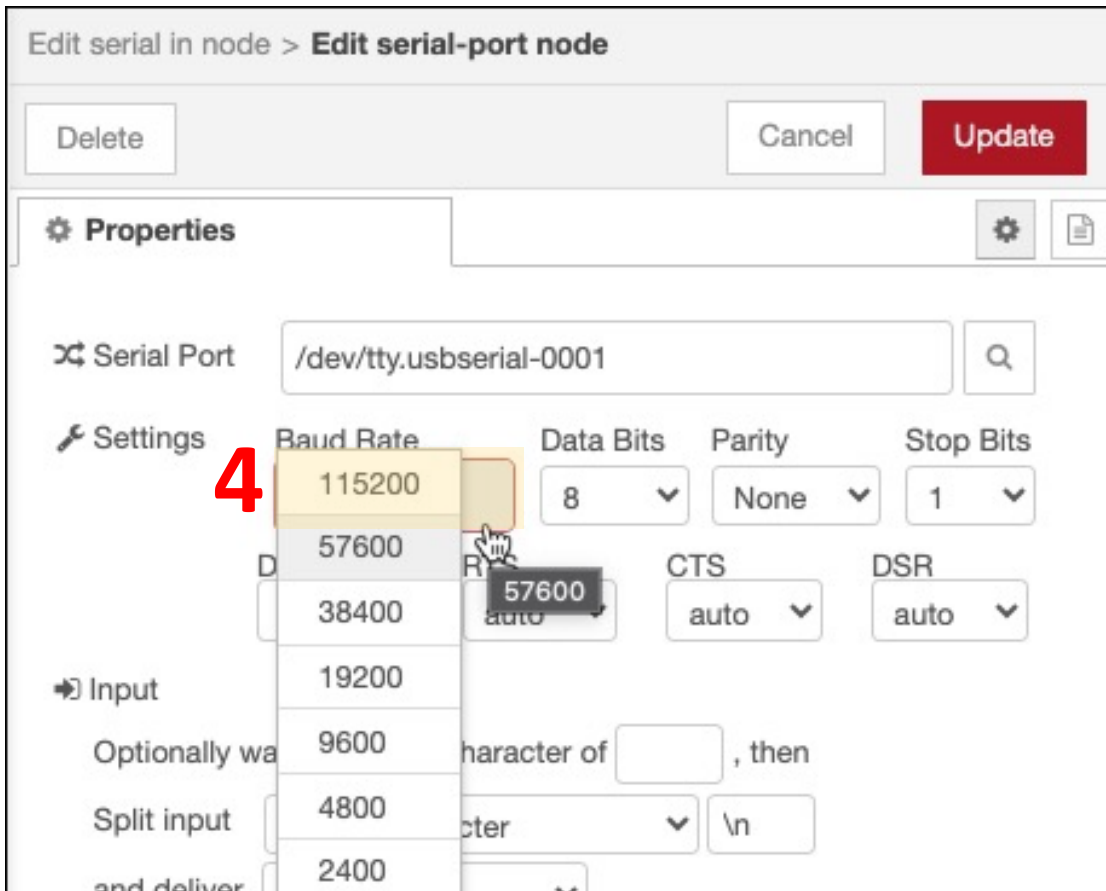
1. Double click **serial** node to configure.
- 2. Click pencil icon to Edit serial-port node.**
3. Click **browse icon** and select the port your EPS32 is connected to.
4. Change the **baud rate** to **115200** or refer to `ESP32 Serial.begin()`.
5. Click **Update button**.
6. Click **Done button**.
7. Click **Deploy button**.

iii. Serial node to debug node



1. Double click **serial** node to configure.
2. Click **pencil icon** to Edit serial-port node.
3. Click **browse icon** and select the port your ESP32 is connected to.
4. Change the **baud rate** to **115200** or refer to `ESP32 Serial.begin()`.
5. Click **Update** button.
6. Click **Done** button.
7. Click **Deploy** button.

iii. Serial node to debug node



1. Double click **serial** node to configure.
2. Click **pencil icon** to Edit serial-port node.
3. Click **browse icon** and select the port your EPS32 is connected to.
4. Change the **baud rate** to **115200** or refer to slide 2. .
5. Click **Update** button.
6. Click **Done** button.
7. Click **Deploy** button.

iii. Serial node to debug node

Edit serial in node > Edit serial-port node

Delete Cancel Update

Properties

Serial Port /dev/tty.usbserial-0001

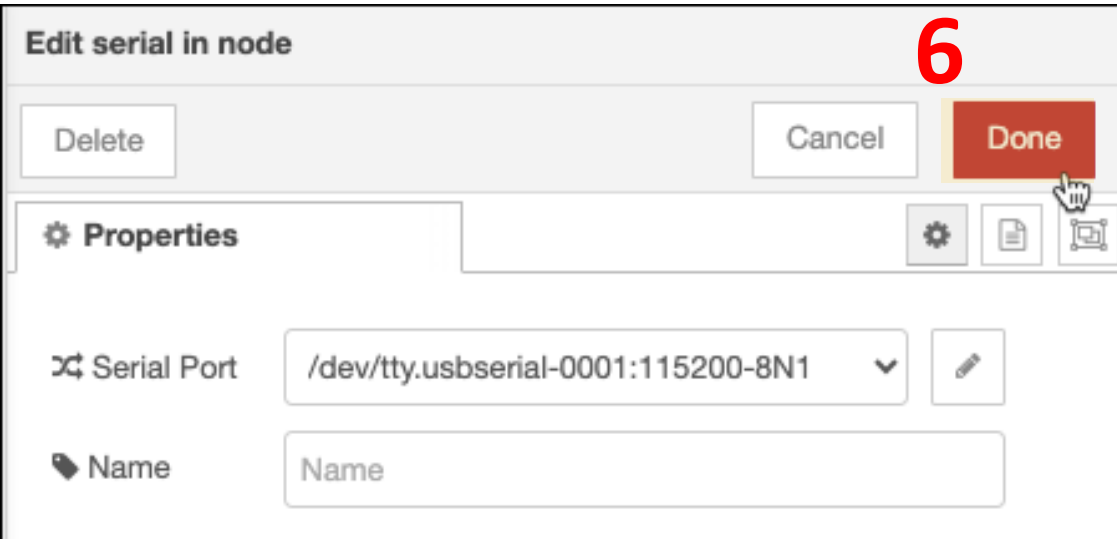
Settings

Baud Rate 115200 Data Bits 8 Parity None Stop Bits 1

DTR auto RTS auto CTS auto DSR auto

1. Double click **serial** node to configure.
2. Click **pencil icon** to Edit serial-port node.
3. Click **browse icon** and select the port your EPS32 is connected to.
4. Change the **baud rate** to **115200** or refer to ESP32 Serial.begin().
5. Click **Update button**.
6. Click **Done button**.
7. Click **Deploy button**.

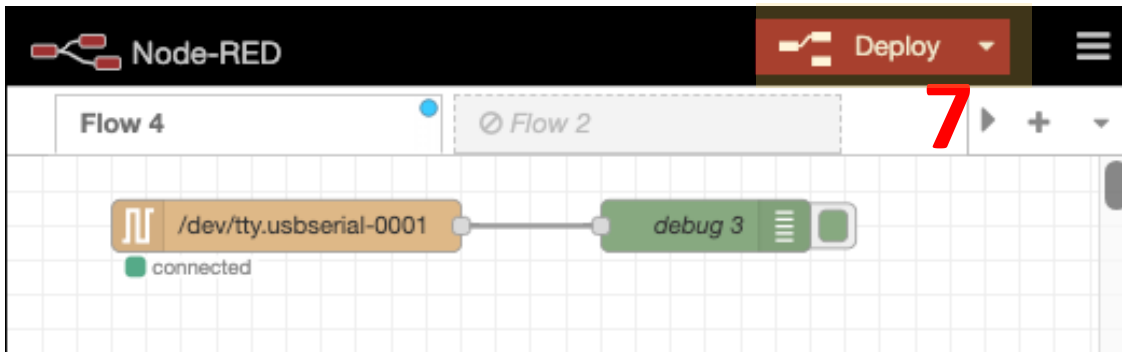
iii. Serial node to debug node



The screenshot shows the 'Edit serial in node' dialog box in Node-RED. At the top, there are three buttons: 'Delete', 'Cancel', and 'Done'. A large red number '6' is overlaid on the 'Done' button, and a mouse cursor is clicking it. Below the buttons is a 'Properties' section with a gear icon. Under 'Serial Port', there is a dropdown menu showing '/dev/tty.usbserial-0001:115200-8N1' and a pencil icon to its right. Below that is a 'Name' field with the placeholder text 'Name'.

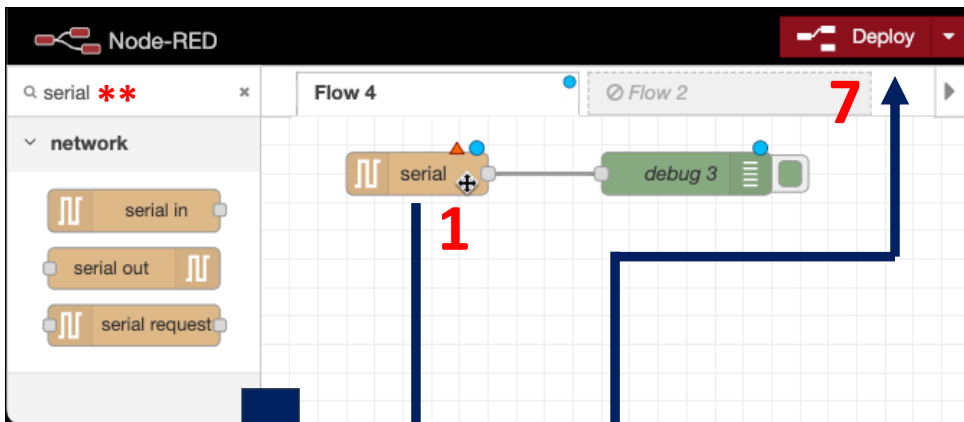
1. Double click **serial** node to configure.
2. Click **pencil icon** to Edit serial-port node.
3. Click **browse icon** and select the port your EPS32 is connected to.
4. Change the **baud rate** to **115200** or refer to ESP32 Serial.begin().
5. Click **Update button**.
6. Click **Done button**.
7. Click **Deploy button**.

iii. Serial node to debug node

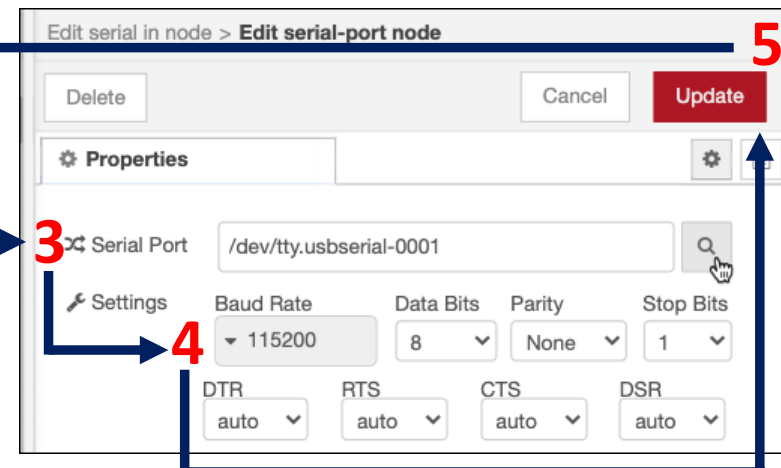
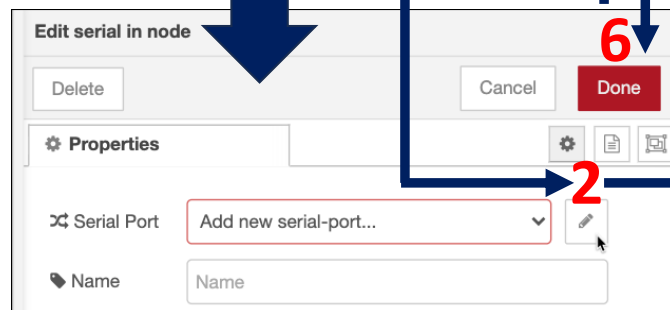


1. Double click **serial** node to configure.
2. Click **pencil icon** to Edit serial-port node.
3. Click **browse icon** and select the port your EPS32 is connected to.
4. Change the **baud rate** to **115200** or refer to ESP32 Serial.begin().
5. Click **Update button**.
6. Click **Done button**.
7. Click **Deploy button**.

iv. Project process flow



1. Double click **serial** node to configure.
2. Click **pencil icon** to Edit serial-port node.
3. Click **browse icon** and select the port your EPS32 is connected to.
4. Change the **baud rate** to **115200** or refer to ESP32 Serial.begin() .
5. Click **Update**.
6. Click **Done** .
7. Click **Deploy** .



****Palette: node-red-node-serialport
ver 0.14.1**

v. Click debug tab (ladybird image) to view the output

The screenshot shows the Node-RED web interface in a browser. The main workspace displays a flow with a serial port node labeled `/dev/tty.usbserial-0001` and a `debug 3` node. A status indicator below the serial port node shows `** connected`. The right-hand sidebar has the `debug` tab selected, indicated by a blue arrow. The debug console shows a list of messages with timestamps and node IDs. The first message's payload is `"31,76.0"`, with a blue arrow pointing to it and the text "Temperature , Humidity".

Timestamp	Node	Payload
12/05/2025, 16:01:36	node: debug 3	<code>msg.payload : string[7]</code> <code>"31,76.0"</code>
12/05/2025, 16:01:37	node: debug 3	<code>msg.payload : string[7]</code> <code>"31,76.0"</code>
12/05/2025, 16:01:39	node: debug 3	

****Not connected indicates other software is currently access the serial port.**

Check your Arduino Ide Serial Monitor



vi. Script to 2 split the values to individual debug

The screenshot shows the Node-RED web interface. A flow named 'Flow 4' contains a serial port node connected to a 'function 3' node, which is then connected to three debug nodes (debug 3, debug 4, debug 5). A large blue arrow points from the 'function 3' node in the flow to the 'Edit function node' dialog. The dialog has three tabs: 'Setup', 'On Start', and 'On Message'. The 'On Message' tab is selected, showing a JavaScript script. A blue arrow points from the script to the 'Setup' tab, which is then shown with the 'Outputs' set to 2. Another blue arrow points from the 'Done' button in the 'Setup' tab to the 'Deploy' button in the top right of the Node-RED interface.

1. Double click **function** node to configure.

2. By default, **On Message** tab is selected.

3. Type the given script inside the editor.

4. Click **Setup** tab and change the **Outputs** from 1 to 2: for temperature & humidity.

5. Click **Done**.

6. Click **Done**.

7. Click **Deploy**.

8. Deploy button in the top right of the Node-RED interface.

1. Double click **function** node to configure.
2. By default, **On Message** tab is selected.
3. Type the given script inside the editor.
4. Click **Setup** tab and change the **Outputs** from 1 to 2: for temperature & humidity.
5. Click **Done**.
6. Click **Done**.
7. Click **Deploy**.

```
var output = msg.payload.split(",");
msg1={};
msg2={};
var read1 = parseInt(output[0]);
var read2 = parseInt(output[1]);

msg1 = {payload : read1;
msg1.topic = 'Temperature';

msg2 = {payload : read2;
msg2.topic = 'Humidity';

return [msg1,msg2];
```

END